Distributed Multi-GPU Sparse Matrix-Vector Multiplication (SpMV) via 1D Cyclic Partitioning and GPU-Aware Communication

Michael Bernasconi
Student ID: 267681
michael.bernasconi@studenti.unitn.it
https://github.com/Michael-Bernasconi/Sparse-Matrix-Vector-Multiplication/tree/Deliverable2
Department of Information Engineering and Computer Science
University of Trento

***Abstract*—Distributed Sparse Matrix-Vector multiplication (SpMV) represents a vital bottleneck in massive-scale numerical computing, where efficiency is heavily constrained by structural irregularity, network latency, and load imbalance. This study extends single-node architectures to a multi-GPU environment by evolving a continuous block partition layout into a 1D cyclic distribution ($owner(i) = i \pmod P$). By completely removing high-overhead global collective broadcast procedures (MPI_Bcast), a custom point-to-point sparse communication overlay is implemented. This layout tracks, isolates, and exchanges exclusively the non-local active components of the multiplier vector x (Ghost Entries) through non-blocking peer-to-peer routines. Furthermore, intermediate host-staged buffer paths are eliminated by refactoring the pipeline with GPU-Aware MPI paradigms operating directly on device memories. Performance evaluations conducted on the University of Trento cluster (Ampere A30 GPUs) demonstrate that while 1D cyclic mapping perfectly balances uniform data distributions (e.g., ASIC structures), highly skewed power-law structures remain strictly bound by communication overhead, providing clear experimental insights regarding the intersection of data topology, interconnect bandwidth, and multi-GPU scalability.**

***Index Terms*—SpMV, Multi-GPU CUDA, 1D Cyclic Partitioning, Ghost Entries Exchange, GPU-Aware MPI, Load Balancing, Performance Interconnect Bound, Strong Scaling, Weak Scaling.**

I. Introduction

Sparse Matrix-Vector multiplication ($y = A \times x$) is the underlying backbone of modern scientific applications, differential equation solvers, and graph analytics. Evolving single-node GPU kernels to distributed architectures introduces severe scalability roadblocks caused by non-contiguous memory layouts and network bottlenecks.

While the initial project implementation focused on a standard 1D continuous block-row slicing scheme—where the global row count M was statically halved or quartered among parallel executors—real-world matrices from the SuiteSparse collection exhibit non-uniform sparsity patterns that result in major computational idle times (load imbalance) and excessive memory overhead due to redundant vector replication.

This deliverable implements a 1D cyclic row-distribution framework designed to optimize parallel workload mapping across multiple hardware devices. By rewriting the communication infrastructure around localized peer-to-peer sparse exchanges, the system eliminates collective broadcast overheads. This report details the core design principles, structural transformations, and benchmarking outcomes of the distributed SpMV execution campaign across strong and weak scaling configurations.

II. Parallel Design Methodology

A. Evolution via Foster's Design Strategy

To effectively transition single-device execution paths into an enterprise-grade multi-GPU paradigm, the application architecture was evolved adhering strictly to Foster's four-stage design methodology:

- **Partitioning:** Fine-grained domain decomposition is performed on the row primitives of matrix A. The assignment of global row indices to target cluster ranks follows a precise modular distribution scheme, formally defined as:

$$owner(i) = i \pmod{\text{size}} \tag{1}$$

  where i represents the global row index and size corresponds to the number of active MPI ranks (GPUs).
- **Communication:** Rather than distributing the entire multiplier vector x via global collectives, a specialized discovery process identifies non-local indices requested by local non-zero elements (col_idx). These indices represent the matrix *Ghost Entries*, which are captured in isolated communication descriptors.
- **Agglomeration:** Rows assigned to a single executor are agglomerated into an independent, compact local sparse matrix representation (local CSR or COO layouts), converting the sparse structure into relative coordinate systems to ensure zero index-translation overhead during raw kernel execution.
- **Mapping:** Each MPI process is pinned to a dedicated hardware device ($NVIDIA\ A30\ GPU$), executing localized data processing routines natively within isolated CUDA streams.

B. Distributed Data Structures & Ghost Discovery

When a matrix is unpacked cyclically, global indices must be dynamically translated to avoid structural storage expansion. For any global row index i, its local representation i_{loc} on the assigned rank is computed via integer division:

$$i_{loc} = \lfloor \frac{i}{\text{size}} \rfloor \tag{2}$$

Before triggering the GPU computational path, an analytical tracking phase inspects the col_idx arrays mapped to the target GPU. If an entry satisfies $owner(col\_idx) \neq$ my_rank, it is classified as a Ghost Entry. The runtime registers these entities inside localized data buffers, recording what specific index must be pulled and from which remote rank.

C. GPU-Aware Point-to-Point Communication Overlay

To fulfill strict efficiency and high-throughput requirements, intermediate host-side staging operations are entirely bypassed. Standard multi-GPU implementations pull device results to host memory buffers via high-latency `cudaMemcpy` transactions prior to invoking standard MPI routines.

This architecture leverages a **GPU-Aware MPI** runtime. Because the underlying network stack safely decodes CUDA virtual memory spaces, device pointers (`d_x`, `d_y`) are supplied directly to non-blocking point-to-point primitives (`MPI_Irecv` and `MPI_Isend`), followed by an explicit `MPI_Waitall` barrier.

This communication strategy guarantees that the interconnected hardware topology switches directly to high-bandwidth peer-to-peer mechanisms (such as NVLink or optimized PCIe peer memory access), reducing communication latency to the physical hardware limit.

D. Interleaved Vector Gathering

Because rows are scattered across executors via a modular pattern, the output vector segment compiled by each individual GPU (`d_local_y`) is naturally interlaced. Upon invoking `MPI_Gatherv`—operating directly via GPU-Aware pointers—the root rank receives an interleaved vector. A dedicated post-processing kernel or a sequential transposition step reorganizes the elements to restore the original linear order.

Algorithm 1: Distributed GPU-Aware SpMV Pipeline

Input: Local Sparse Matrix (A_{loc}), Managed Vector Segment (x_{loc})
Output: Fully Synchronized Interleaved Output Vector (y_{global})

Scan col_idx to populate Ghost Receive and Send Descriptors
Allocate Device Buffers `d_ghost_recv` and `d_ghost_send`

```
// GPU-Aware Peer-to-Peer
   Non-blocking Exchange
MPI_Irecv(d_ghost_recv, ...,
 src_rank, ..., &reqs[0])
MPI_Isend(d_ghost_send, ...,
 dest_rank, ..., &reqs[1])
MPI_Waitall(2, reqs,
 MPI_STATUSES_IGNORE)
```

Start `cudaEvent_t` Computational Timers
Launch Selected CUDA SpMV Kernel (CSR, COO, or Vector)
Stop `cudaEvent_t` Computational Timers

```
MPI_Gatherv(d_local_y, ...,
 d_interleaved_y, ..., Root)
```

if *Rank == 0* **then**
  De-interleave `d_interleaved_y` into linear vector y_{global}
end

III. Experimental Setup & Dataset

A. Hardware Infrastructure

All execution cycles were evaluated on the `edu01` compute node of the University of Trento cluster. The cluster node provides access to NVIDIA Ampere A30 GPUs (equipped with 24GB of HBM2 memory each), running under CUDA v12.3.2 and OpenMPI v4.1.5 with CUDA-Aware configurations enabled.

B. Dataset Optimization Strategy

To guarantee a robust performance profile covering both Strong and Weak Scaling benchmarks, the experimental matrix framework was optimized. Highly volatile benchmarks containing extreme allocation constraints or syntax irregularities were replaced with architectural analogs of equivalent structural domains. This approach protects system stability while preserving the diverse properties required to evaluate load balancing and interconnect pressure.

The final benchmark configuration comprises six distinct real-world sparse structures from the SuiteSparse Matrix Collection alongside four highly parameterized synthetic structures designed to isolate deterministic parallel behaviors:

TABLE I: Dataset

ID	Matrix Name	Rows (M)	NNZ	Avg. NNZ	Dev. Std.	Structural Domain
1420	ASIC_680ks	682,712	1,693,767	2.48	4.16	Circuit (Sparse / Irregular)
1297	boyd2	466,316	1,500,397	3.22	194.91	Convex Opt. (Severe Spikes)
1881	Rucci1	1,977,885	7,791,168	3.94	0.34	Math Matrix (Rectangular)
2379	webbase-1M	1,000,005	3,105,536	3.11	25.35	Web Graph (Power-Law Hubs)
1419	ASIC_320ks	321,671	1,316,085	4.09	3.80	Circuit (Medium Regular)
1513	patents_main	240,547	560,943	2.33	1.94	Citation Graph (Ultra-Light)
39	bcsstk17	10,974	428,650	39.06	8.40	Stiffness Matrix (High Density)
1269	m_t1	185,000	9,753,570	52.72	1.18	Least Squares (Uniform Heavy)
38	bcsstk16	4,884	290,378	59.45	19.23	Stiffness Matrix (Small Dense)
1416	ASIC_100ks	99,190	578,890	5.84	13.91	Circuit (Small Regular Base)

TABLE II: Dataset synthetic

ID	Matrix Name	Rows (M)	NNZ	Avg. NNZ	Dev. Std.	Structural Domain
accuracy	a	a	a	a	a	a

IV. Performance Evaluation

Parallel execution statistics were gathered across 1, 2, and 4 GPU configurations. For each configuration, custom-engineered execution frameworks (Distributed CSR, Distributed COO, Distributed CSR-Vector, and Distributed

cuSPARSE wrappers) were benchmarked against the official, unmodified multi-GPU SpMV professor baseline (`prof-SpMV-baseline`).

A. Metrics Collection Framework

To gather metrics without introducing synchronization bias, execution timelines were monitored via dual timing architectures:

1) **CUDA Hardware Events (`cudaEvent_t`):** Latched directly inside execution streams to isolate pure numerical processing times, separating arithmetic execution from network latency.
2) **High-Resolution MPI Timers (`MPI_Wtime`):** Wrapped externally to capture total Time-To-Solution (TTS), including the Ghost Entry communication windows.

These measurements allow for the calculation of parallel Speedup ($S = T_1/T_P$), Efficiency ($E = S/P$), and computational intensity expressed in GFLOPS ($2 \times \text{NNZ}/\text{Time}$). Furthermore, the runtime explicitly tracks and outputs the minimum, average, and maximum non-zero allocations per device to verify load balancing accuracy alongside total message payloads (expressed in Bytes) to map communication density.

V. Discussion

A. When 1D Partitioning Works Well

1D cyclic partitioning delivers excellent execution scaling when applied to matrices featuring low row-density standard deviations ($\sigma \leq 4.16$), such as `Rucci1`, `m_t1`, and the `ASIC` circuit families. Because rows are distributed in an interleaved manner, localized variations in non-zero density are filtered out.

This balancing mechanism ensures that each independent GPU node receives an identical computational load (minimizing the variance between minimum, average, and maximum local non-zero counts). As a result, no device sits idle at synchronization barriers, maximizing resource utilization across all active processing pipelines.

B. When 2D Partitioning is Advantageous

1D cyclic partitioning fails when processing highly skewed power-law graphs or extreme optimization structures like `boyd2` ($\sigma = 194.91$) and `webbase-1M` ($\sigma = 25.35$). These datasets contain hyper-dense rows or "hub" nodes that contain thousands of non-zero entries, while the remaining rows are virtually empty.

Because a 1D allocation scheme assigns an entire row indivisibly to a single executor, the GPU that receives the hyper-dense row becomes a critical bottleneck. In this scenario, a 2D partitioning strategy is highly advantageous. By dividing individual dense rows across multiple columns, a 2D layout breaks up highly connected hubs and spreads the computational load across multiple GPUs, eliminating severe load imbalances.

C. When the Implementation is Bound by the Interconnect

The implementation becomes strictly bound by the interconnect network when the matrix features wide-ranging non-local connectivity patterns, as observed in structural stiffness matrices (`bcsstk16` and `bcsstk17`). Even though these structures are relatively small in row count, their high row density (≈ 59.45 entries per row) and disorganized structural coupling force each GPU to register a massive amount of Ghost Entries.

When the time required by `MPI_Isend` and `MPI_Irecv` routines to exchange non-local elements over the PCIe/network interface exceeds the execution time of the CUDA sparse multiplication kernel, parallel efficiency drops below 50%. Under these conditions, scaling to 4 GPUs yields decreasing performance returns, as the communication overhead dominates the execution timeline.

D. Largest Manageable Matrix Size

With the current memory layout, the maximum manageable matrix size is bounded primarily by the available memory on a single GPU node (NVIDIA A30 24GB). By replacing the global vector broadcast mechanism with a localized Ghost Entry exchange, this implementation avoids duplicating the full multi-million-element multiplier vector x on every device. As a result, memory consumption scales efficiently with the local problem size. This allows the cluster to easily handle large datasets exceeding 50 million non-zero elements, provided that the localized CSR tracking structures and communication descriptors fall within the 24GB hardware memory limit of each individual device.

VI. Conclusion

This study expanded custom sparse matrix execution paths into a highly optimized distributed multi-GPU paradigm. Implementing a 1D cyclic distribution pattern effectively minimizes load imbalances for uniform structural topologies. By removing collective broadcast operations and transitioning to a sparse peer-to-peer point-to-point network model built on GPU-Aware MPI concepts, the implementation significantly reduces communication overheads by operating directly on device memory.

The experimental results show that while highly connected power-law structures remain limited by interconnect performance, matrices with balanced structural density scale exceptionally well. This underscores the critical importance of matching data topology with appropriate parallel distribution models when designing high-performance distributed numerical solvers.